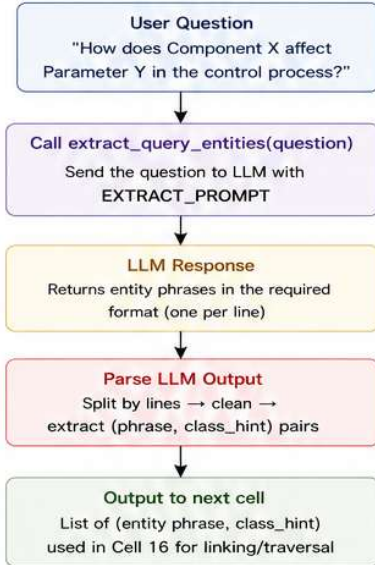


CELL 15 – QUERY ENTITY EXTRACTION

Purpose: The only LLM call before retrieval. It does NOT retrieve an answer.
It only finds the entity phrases in the question so the KG can look them up in Cell 16.

1) OVERALL WORKFLOW



4) LLM EXPECTED OUTPUT FORMAT (example)

Component X|Component
Parameter Y|ControlFactor
control process|Procedure
ISO 26262|Standard
NONE

2) THE EXTRACT_PROMPT (what the LLM sees)

EXTRACT_PROMPT = "" "You extract entity mentions from a question so they can be looked up in a systems-engineering knowledge graph.

Return every noun phrase in the question that names a concrete thing: a system, component, parameter, method, procedure, standard, document, or failure mode. Use the exact phrase as it appears in the question.

Do NOT return: question words, verbs, generic words like "process" or "information", or anything you invent that is not in the question.

Output one record per entity, formatted exactly as:

<entity phrase>|<one of: System, Component, ControlFactor, NoiseFactor, ErrorState, Method, Procedure, Standard, Concept>

No preamble, no markdown, no numbering. Only the records.

If the question names no concrete entity at all, output exactly: NONE""

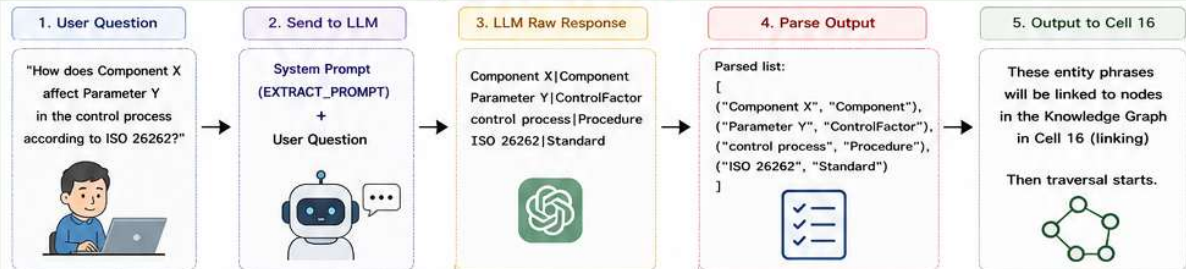
3) FUNCTION: extract_query_entities(question)

```
def extract_query_entities(question: str):
    """-> [(phrase, class_hint)]. Never raises; an LLM failure degrades
    to [] and the caller falls back to lexical matching in Cell 16."""
    try:
        # Call the LLM once
        resp = driver_llm.invoke([
            {"role": "system", "content": EXTRACT_PROMPT},
            {"role": "user", "content": question},
        ])
        text = resp.content.strip() # get plain text reply
    except Exception as e:
        print(f"entity extraction failed ({e}) -- lexical fallback")
        return [] # safe fallback

    if text.upper().startswith("NONE"):
        return [] # no concrete entity found

    out = []
    for line in text.splitlines():
        line = line.strip().rstrip("-*0123456789. ").strip()
        if line and "|" in line:
            phrase, cls = line.split("|", 1)
            if phrase.strip():
                out.append((phrase.strip(), cls.strip()))
    return out
```

5) END-TO-END EXAMPLE



6) LINE-BY-LINE CODE EXPLANATION

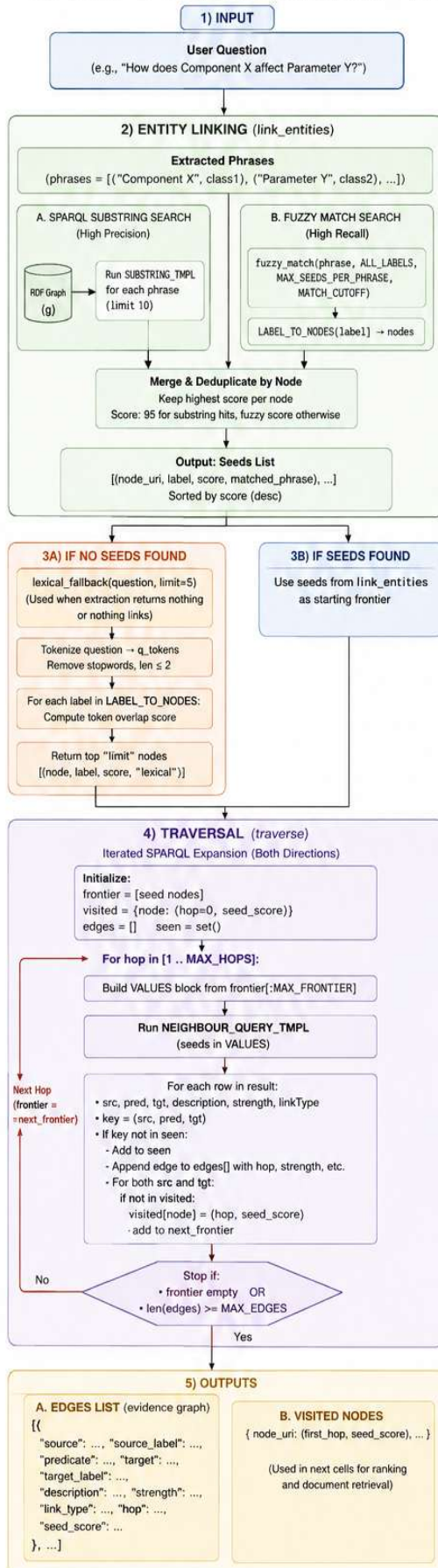
Code	What it does (simple explanation)
<code>resp = driver_llm.invoke([...])</code>	Sends the system prompt and user question to the LLM and gets the response.
<code>text = resp.content.strip()</code>	Takes the text reply from LLM and removes leading/trailing spaces.
<code>except Exception as e: ... return []</code>	If the LLM call fails (timeout, API error, etc.), return empty list. The system will use lexical fallback later (Cell 16).
<code>if text.upper().startswith("NONE"):</code>	If the LLM says NONE, it means the question has no concrete entity. Return empty list.
<code>for line in text.splitlines():</code>	Process each line of the LLM output (LLM returns one entity per line).
<code>line.strip().rstrip("-*0123456789. ").strip()</code>	Remove bullets, numbers, dashes, and spaces from the start and end.
<code>if line and " " in line:</code>	A valid record must contain a pipe ' ' separating phrase and class.
<code>phrase, cls = line.split(" ", 1)</code>	Split into two parts: entity phrase and class (only the first pipe is used).
<code>out.append((phrase.strip(), cls.strip()))</code>	Clean spaces and add the pair to the output list.
<code>return out</code>	Return final list of (entity phrase, class_hint).

7) KEY POINTS

- This cell only identifies entity mentions; it does NOT fetch any information from the KG.
- Output is a clean list of (entity phrase, class_hint) used as seed candidates in Cell 16.
- If LLM fails or returns NONE, the system safely continues using lexical matching in Cell 16.
- Keeping this step simple and strict → better linking quality → better retrieval.

CELL 16 – LINKING + LINKRETRIAL (SPARQL, no LLM)

Goal: From query → link entities (SPARQL substring + fuzzy) → seed nodes → iterated SPARQL traversal (both directions) up to MAX_HOPS → edges + visited nodes



GLOBAL CONSTANTS & HELPERS (from code)

```
_STOP = {"the", "a", "an", "of", "for", "and", "or", "to", "in", "on",
        "is", "are", "what", "which", "how", "does", "do", "with",
        "by", "from"}

def _tokens(s):
    return [t for t in re.findall(r"[a-z0-9]+", (s or "").lower())
            if t not in _STOP and len(t) > 2]

def _values_block(uris):
    return " ".join(f "<u>" for u in uris
                    if "<" not in u and ">" not in u)

def _sparql_literal(s):
    return " " + s.replace("\\", "\\\").replace("'", "\\'") + " "
```

SPARQL TEMPLATES USED

```
1) NEIGHBOUR_QUERY_TMPL (used in traverse for each hop)
SELECT ?src ?srcLabel ?pred ?predLabel ?tgt ?tgtLabel
      ?description ?strength ?linkType
WHERE {
  VALUES ?seed { %SEEDS% }
  ?stmt a rdf:Statement ;
    rdf:subject ?src ;
    rdf:predicate ?pred ;
    rdf:object ?tgt ;
    ex:description ?description ;
    ex:strength ?strength ;
    ex:linkType ?linkType .
  FILTER(?src = ?seed || ?tgt = ?seed)
  OPTIONAL { ?src rdfs:label ?srcLabel }
  OPTIONAL { ?tgt rdfs:label ?tgtLabel }
  OPTIONAL { ?pred rdfs:label ?predLabel }
}

2) SUBSTRING_TMPL (used in link_entities substring pass)
SELECT DISTINCT ?node ?label
WHERE {
  ?node rdfs:label ?label .
  FILTER(CONTAINS(LCASE(STR(?label)), %PHRASE%))
  FILTER(!STRSTARTS(STR(?node), "%PREDMARK%"))
}
LIMIT 10

3) DOC_CONTENT_TMPL (used in later cells to fetch docs)
SELECT ?doc ?label ?content
WHERE {
  VALUES ?doc { %DOCS% }
  ?doc ex:documentContent ?content .
  OPTIONAL { ?doc rdfs:label ?label }
```

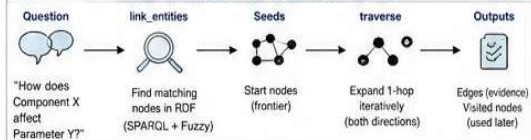
CORE FUNCTIONS (from code)

```
1) link_entities(phrases)
Input: phrases = [{"phrase", class}, ...]
- For each phrase:
  a) Run SUBSTRING_TMPL (SPARQL) → candidates (score=95)
  b) Run fuzzy_match → candidates
- Merge results by node (keep highest score)
- Return [(node, label, score, matched_phrase), ...] sorted

2) lexical_fallback(question, limit=5)
- Tokenize question → q_tokens
- For each label in LABEL_TO_NODES:
  - overlap = q_tokens ∩ tokens(label)
  - score = 100 * |overlap| / max(|tokens(label)|, 1)
- Return top "limit" nodes as [(node, label, score, "lexical")]

3) traverse(seeds)
Input: seeds = [(node, label, score, phrase), ...]
- frontier = seed nodes
- visited = {node: (hop=0, seed_score)}
- For hop in 1..MAX_HOPS:
  a) Run NEIGHBOUR_QUERY_TMPL with frontier in VALUES
  b) For each row:
    - Add edge if not seen (src, pred, tgt)
    - Update visited and next_frontier
  c) frontier = next_frontier
  d) Stop if frontier empty or len(edges) ≥ MAX_EDGES
```

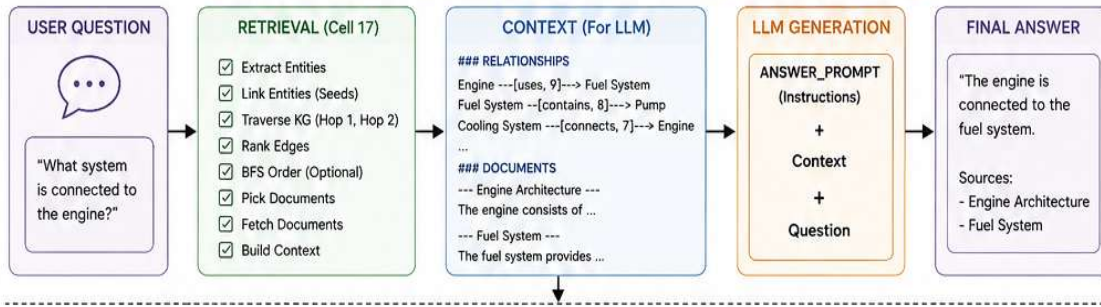
HOW IT WORKS – END TO END EXAMPLE



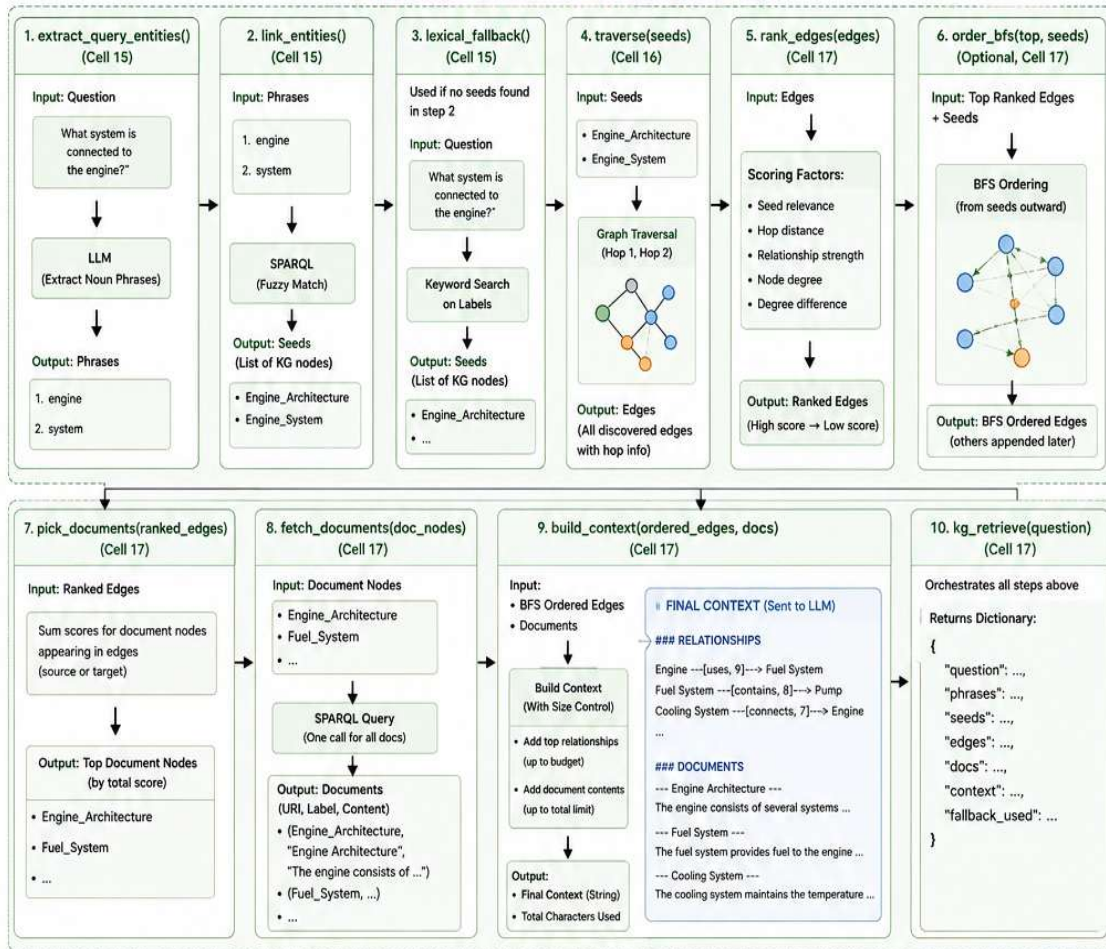
KEY POINTS

- ✓ No LLM, no embeddings – purely lexical + SPARQL graph exploration.
- ✓ Iterated traversal (1-hop per query) for performance & flexibility.
- ✓ Both directions: seed can be subject OR object in reified statement.
- ✓ Edges are reified statements with description, strength, linkType.

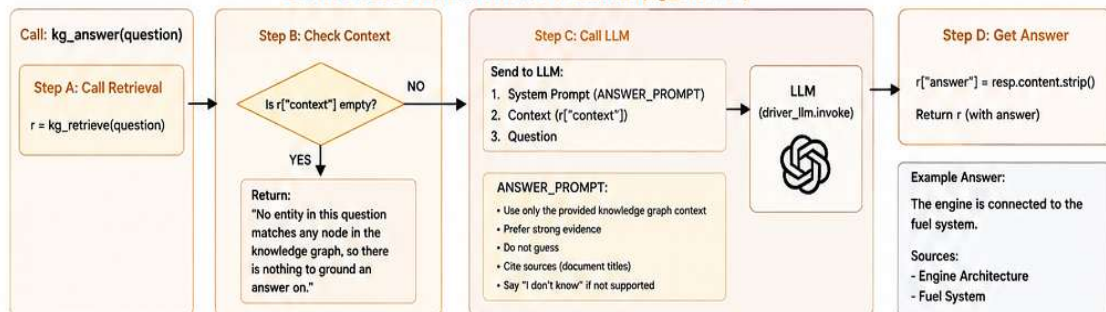
END-TO-END PIPELINE DIAGRAM – COMPLETE VIEW



1. DETAILED RETRIEVAL PIPELINE (Cell 17)



2. ANSWER GENERATION PIPELINE (kg_answer)



IMPORTANT NOTES

- Ranking decides WHICH edges to keep.
- BFS decides HOW to present them.
- Documents are picked based on total score from ranked edges.
- Context size is strictly controlled to fit LLM limits.
- Retrieval and generation are separated for debugging and transparency.

KEY PARAMETERS (From Cell 11)

MAX_SEARCH_TERMS = 6
 MAX_SEEDS = 25
 MAX_HOP1 = 60
 MAX_HOP2 = 180
 MAX_TRIPLES = 30
 MAX_DOCS = 5
 MAX_CONTEXT_CHARS = 20000
 MAX_DOC_CHARS = 8000
 TRIPLE_BUDGET_FRAC = 0.30

DATA FLOW SUMMARY

1. Question → Entities → Seeds
2. Seeds → Traversal → Edges
3. Edges → Ranking → Top Edges
4. Top Edges → BFS Ordering
5. Ranked Edges → Pick Docs → Fetch Docs
6. Ordered Edges + Docs → Build Context
7. Context + Question + Prompt → LLM
8. LLM → Final Answer

COMPLETE FUNCTIONS (Cell 17)

- ✓ rank_edges(edges)
- ✓ order_bfs(top_edges, seeds)
- ✓ pick_documents(ranked_edges)
- ✓ fetch_documents(doc_nodes)
- ✓ build_context(ordered_edges, docs)
- ✓ kg_retrieve(question)
- ✓ kg_answer(question)

CELL 19 : STRICT KG-ONLY TOP-10 CHUNK RETRIEVER (CODE-WISE FLOW)

NO LLM • NO BM25 • NO embeddings • NO vector search
RANKING = seed_score x ex:strength x hop decay (100% graph-derived)

CODE (SECTION)

```
TOP_K = 10
CHUNK_SIZE = 900
MIN_CHUNK = 120
MAX_PER_DOC = 3
JACCARD = 0.25
SHINGLE_N = 5

_clean(t)
_shingles(t, n=SHINGLE_N)
_split_chunks(text, size=CHUNK_SIZE)
```

CODE (SECTION)

```
# ---- Chunk corpus, keyed by document URI ----
DOC_Q = PREFIXES + """
SELECT ?doc ?label ?ref ?content
WHERE {
  ?doc ex:documentContent ?content .
  OPTIONAL { ?doc rdfs:label ?label }
  OPTIONAL { ?doc ex:sourceReference ?ref }
}
"""

CHUNKS_BY_DOC = defaultdict(list)
for row in g.query(DOC_Q):
    uri = str(row.doc)
    title = str(row.label) if row.label
    else NODE_LABEL.get(uri, uri)
    ref = str(row.ref) if row.ref else uri
    for ch in _split_chunks(_clean(str(row.content))):
        CHUNKS_BY_DOC[uri].append({"content": ch,
                                     "title": title, "reference": ref, "doc_uri": uri})
```

CODE (SECTION)

```
# ---- Step 1: KG lexical node matching ----
seeds = lexical_fallback(QUERY_KG, limit=10)

# seeds -> list of tuples:
# (node_uri, label, seed_score, matched_tokens)
```

CODE (SECTION)

```
# ---- Step 2: SPARQL multi-hop traversal ----
edges, visited = traverse(seeds)

# edges -> list of dicts:
# {source, target, predicate, strength,
#  hop, seed_score}
# visited -> dict {node_uri: (min_hop, seed_score)}
```

CODE (SECTION)

```
# ---- Step 3: graph evidence per document ----
doc_evidence = defaultdict(float)
for e in edges:
    w = (e["strength"] / 10.0) *
        (HOP_DECAY ** (e["hop"] - 1)) *
        (e["seed_score"] / 100.0)
    doc_evidence[e["source"]] += w
    doc_evidence[e["target"]] += w

for uri, lbl, score, p in seeds:
    doc_evidence[uri] = max(
        doc_evidence[uri], score / 100.0)

scoped_docs = {d for d in doc_evidence
                if d in CHUNKS_BY_DOC}
```

CODE (SECTION)

```
# ---- Step 4: rank chunks by doc evidence ----
scored = []
for uri in sorted(scoped_docs,
                  key=lambda d: doc_evidence[d],
                  reverse=True):
    ev = doc_evidence[uri]
    for c in CHUNKS_BY_DOC[uri]:
        scored.append({"c": c,
                       "score": round(ev, 4),
                       "graph_evidence": round(ev, 4)})

scored.sort(key=lambda x: x["score"], reverse=True)
```

CODE (SECTION)

```
# ---- Step 5: dedup + per-doc cap -> top K ----
chosen, seen_h, seen_sh, per_doc = [], [], [], defaultdict(int)
for c in scored:
    if len(chosen) >= TOP_K:
        break
    if per_doc[c["reference"]] >= MAX_PER_DOC:
        continue
    h = hashlib.md5(c["content"].encode()).hexdigest()
    if h in seen_h:
        continue
    sh = _shingles(c["content"])
    if sh and any(sh & p["shingles"] for p in seen_sh):
        continue
    chosen.append(c); seen_h.add(h); seen_sh.append(sh)
    per_doc[c["reference"]] += 1
```

CODE (SECTION)

```
# ---- Response payload ----
response_kg = {
    "data": [
        {
            "content": c["content"],
            "title": c["title"],
            "reference": c["reference"]}
        for c in chosen
    ]
}

# Print summary + json.dumps(response_kg, ...)
```

INPUT QUERY

QUERY_KG = ("What steps should be taken if an Engineering Supplier Integration Document (ESID) is not already available for a supplier?")

USES / DEPENDS ON

- PREFIXES, NODE_LABEL
- _tokens, HOP_DECAY, MAX_HOPS
- lexical_fallback(), traverse()
- from Cell 16

STEP 0: PARAMETERS & HELPER FUNCTIONS

(Text cleaning, shingle generation, sentence-boundary chunking)

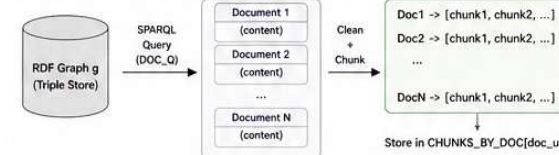
Helper	Purpose
_clean(t)	Remove unresolved links like [<doc:wiki:...>] and extra spaces
_shingles(t)	Create set of word shingles (n=SHINGLE_N) for dedup
_split_chunks(text)	Split cleaned text into sentence-boundary chunks (size=CHUNK_SIZE, no overlap, min length MIN_CHUNK)

Output

Constants
+
Helper Functions
ready

STEP 1: CORPUS BUILDING (From RDF Graph g)

Read document content from RDF graph 'g' and chunk it.



Output

CHUNKS_BY_DOC
(dict)

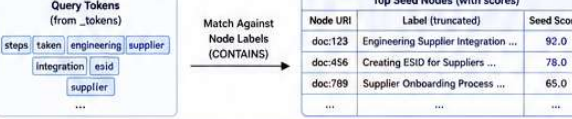
{ doc_uri: { (content,
title, reference,
doc_uri), ... } }

N_CHUNKS
(total chunks)

Print corpus stats

STEP 2: KG LEXICAL NODE MATCHING (No LLM)

Tokenize query and match tokens against node labels (CONTAINS).



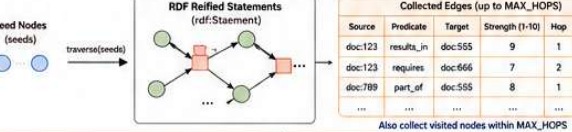
Output

seeds = [
 (uri, label, score, tokens),
 ... (top K)
]

If empty -> STOP

STEP 3: SPARQL MULTI-HOP TRAVERSAL OVER REIFIED STATEMENTS

Use SPARQL over rdf:Statement (iterative per hop up to MAX_HOPS).



Output

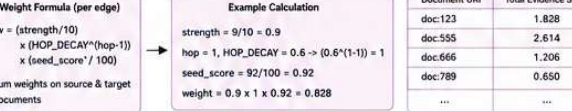
edges (list)

visited (dict)

Print counts

STEP 4: GRAPH EVIDENCE PER DOCUMENT

Aggregate evidence on BOTH endpoints of each edge.



Output

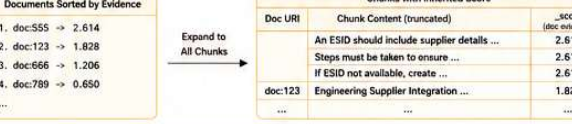
doc_evidence
(dict{doc_uri:score})

scoped_docs (set)

Print count

STEP 5: RANK CHUNKS BY DOCUMENT'S GRAPH EVIDENCE

Every chunk inherits its document's evidence score.



Output

scored (list of chunk dicts)

Each dict has:
content, title, reference,
doc_uri, score,
_graph_evidence

STEP 6: DEDUPLICATION + PER-DOC CAP -> TOP 10

Remove duplicates and near-duplicates, cap per document, take Top 10.



Output

chosen (list, len<=TOP_K)

Top 10 chunk dicts after
dedup & per-doc cap

STEP 7: RESPONSE PAYLOAD (NO LLM)

Return only content, title, reference for top 10 chunks.



Output Variable

response_kg
(dict)

(Used downstream)